

| GUÍA 11: Formularios, validaciones y pasado de información

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1105	Desarrollo de Aplicaciones Móviles	EA2: Desarrollando componentes visuales y funcionalidades nativas para aplicaciones móviles
Tiempo	Modalidad de Trabajo	Indicadores de logro
3 horas	Parejas	IL2.1, IL2.2 y IL2.3



Antecedentes generales

Esta actividad busca que implementes un flujo de registro con formularios reactivos utilizando Jetpack Compose, Material 3 y la arquitectura MVVM. Su propósito es que comprendas cómo modelar los datos de entrada mediante data class, aplicar validaciones reactivas desde el ViewModel y visualizar los datos ingresados en otra pantalla utilizando un ViewModel compartido. Este trabajo fomenta la modularidad, la separación de responsabilidades y el uso del estado como mecanismo central en Jetpack Compose.



Requerimientos para esta actividad

Para el desarrollo de esta actividad deberás disponer de:

- Un proyecto Android Studio nuevo (API 33 o superior)
- Arquitectura MVVM con estructura de carpetas definida
- Jetpack Compose y Material 3 actualizados
- Dependencias de Navigation Compose (2.7.x)
- Conexión a internet para consultar documentación oficial
- Un repositorio GitHub activo y funcional
- Tablero Trello para registrar la tarea por pareja



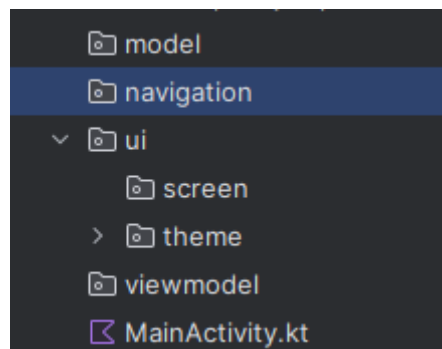
Actividad

En esta actividad práctica desarrollarás un flujo completo de ingreso y visualización de datos personales, aplicando los principios del patrón arquitectónico MVVM y aprovechando la reactividad de StateFlow en Jetpack Compose.

A través de la creación de un formulario de registro de usuario, se espera que validen los datos ingresados en tiempo real y luego los visualicen en una segunda pantalla sin necesidad de pasar argumentos entre rutas, utilizando un ViewModel compartido.

La tarea busca que exploren de manera autónoma la correcta definición de modelos de estado (data class), la manipulación del estado con MutableStateFlow, y la observación reactiva con collectAsState. Además, deberán aplicar principios de jerarquía visual, buenas prácticas de diseño con Material 3 y una organización lógica del código, asegurando la separación de responsabilidades en capas (UI, ViewModel, modelo).

El trabajo se desarrollará en parejas, registrando los avances en GitHub (commits, rama feature/formulario-usuario) y Trello (tarjeta: Formulario con resumen). Deberá presentarse como evidencia el correcto funcionamiento de la validación, la navegación, y la visualización de los datos ingresados.



♦ Parte 1 – Modelado del estado

- Recuerda las dependencias

```
dependencies {
    implementation("androidx.navigation:navigation-compose:2.7.7")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0-rc02")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.6.2")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(dependencyProvider = libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    implementation(libs.androidx.navigation.runtime.android)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}
```



- Crear la clase UsuarioUiState con las propiedades nombre, correo, clave, dirección.

```
// Modelo principal que representa el estado del formulario del usuario
data class UsuarioUiState{
    val nombre: String = "",           // Nombre del usuario
    val correo: String = "",           // Correo electrónico
    val clave: String = "",            // Contraseña
    val direccion: String = "",        // Dirección del usuario
    val aceptaTerminos: Boolean = false, // Confirmación de términos
    val errores: UsuarioErrores = UsuarioErrores() // Objeto que contiene los errores por campo
}
```

- Crear UsuarioErrores para manejar los errores de validación.

```
// Modelo que almacena posibles errores individuales del formulario
2 Usages
data class UsuarioErrores{
    val nombre: String? = null,
    val correo: String? = null,
    val clave: String? = null,
    val direccion: String? = null
}
```

◇ Parte 2 – Lógica del ViewModel

- Implementar UsuarioViewModel usando MutableStateFlow para almacenar el estado.



```
import androidx.lifecycle.ViewModel
import com.example.ejemplo3.model.UsuarioErrores
import com.example.ejemplo3.model.UsuarioUiState
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.update

class UsuarioViewModel : ViewModel() {

    // Estado interno mutable
    8 Usages
    private val _estado = MutableStateFlow(value = UsuarioUiState())

    // Estado expuesto para la UI
    val estado: StateFlow<UsuarioUiState> = _estado

    // Actualiza el campo nombre y limpia su error
    fun onNombreChange(valor: String) {
        _estado.update { it.copy(nombre = valor, errores = it.errores.copy(nombre = null)) }
    }

    // Actualiza el campo correo
    fun onCorreoChange(valor: String) {
        _estado.update { it.copy(correo = valor, errores = it.errores.copy(correo = null)) }
    }

    // Actualiza el campo clave
    fun onClaveChange(valor: String) {
        _estado.update { it.copy(clave = valor, errores = it.errores.copy(clave = null)) }
    }

    // Actualiza el campo dirección
    fun onDireccionChange(valor: String) {
        _estado.update { it.copy(direccion = valor, errores = it.errores.copy(direccion = null)) }
    }

    // Actualiza checkbox de aceptación
    fun onAceptarTerminosChange(valor: Boolean) {
        _estado.update { it.copy(aceptaTerminos = valor) }
    }
}
```

- Incluir funciones de modificación de campos y una función validarFormulario() que retorne true o false y actualice los errores si corresponde.



```
// Validación global del formulario
fun validarFormulario(): Boolean {
    val estadoActual = _estado.value
    val errores = UsuarioErrores(
        nombre = if (estadoActual.nombre.isBlank()) "Campo obligatorio" else null,
        correo = if (!estadoActual.correo.contains(other = "@")) "Correo inválido" else null,
        clave = if (estadoActual.clave.length < 6) "Debe tener al menos 6 caracteres" else null,
        direccion = if (estadoActual.direccion.isBlank()) "Campo obligatorio" else null
    )

    val hayErrores = listOfNotNull(
        ...elements = errores.nombre,
        errores.correo,
        errores.clave,
        errores.direccion
    ).isEmpty()

    _estado.update { it.copy(errores = errores) }

    return !hayErrores
}
```

◇ Parte 3 – Pantalla de Registro

- Crear RegistroScreen con componentes de Material 3: OutlinedTextField, PasswordVisualTransformation, Checkbox, Button.



```
3 import androidx.compose.foundation.layout.Arrangement
4 import androidx.compose.foundation.layout.Column
5 import androidx.compose.foundation.layout.Row
6 import androidx.compose.foundation.layout.Spacer
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.fillMaxWidth
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.foundation.layout.width
11 import androidx.compose.material3.Button
12 import androidx.compose.material3.Checkbox
13 import androidx.compose.material3.MaterialTheme
14 import androidx.compose.material3.OutlinedTextField
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.runtime.collectAsState
18 import androidx.compose.runtime.getValue
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.text.input.PasswordVisualTransformation
22 import androidx.compose.ui.unit.dp
23 import androidx.navigation.NavController
24 import com.example.ejemplo3.viewmodel.UsuarioViewModel
25
26 2 Usages
27 @Composable
28 fun RegistroScreen(
29     navController: NavController,
30     viewModel: UsuarioViewModel
31 ) {
32     ⚡ val estado by viewModel.estado.collectAsState()
33
34     Column(
35         Modifier
36             .fillMaxSize()
37             .padding(all = 16.dp),
38         verticalArrangement = Arrangement.spacedBy(space = 12.dp)
39     ) {
40         // Campo nombre
41         OutlinedTextField(
42             value = estado.nombre,
43             onChange = viewModel::onNombreChange,
```

- Mostrar errores con isError y supportingText.

```
44     label = { Text( text = "Nombre" ) },
45     isError = estado.errores.nombre != null,
46     supportingText = {
47         estado.errores.nombre?.let {
48             Text( text = it, color = MaterialTheme.colorScheme.error)
49         }
50     },
51     modifier = Modifier.fillMaxWidth()
52 )
53
54 // Campo correo
55 OutlinedTextField(
56     value = estado.correo,
57     onValueChange = viewModel::onCorreoChange,
58     label = { Text( text = "Correo electrónico" ) },
59     isError = estado.errores.correo != null,
60     supportingText = {
61         estado.errores.correo?.let {
62             Text( text = it, color = MaterialTheme.colorScheme.error)
63         }
64     },
65     modifier = Modifier.fillMaxWidth()
66 )
67
68 // Campo clave
69 OutlinedTextField(
70     value = estado.clave,
71     onValueChange = viewModel::onClaveChange,
72     label = { Text( text = "Contraseña" ) },
73     visualTransformation = PasswordVisualTransformation(),
74     isError = estado.errores.clave != null,
75     supportingText = {
76         estado.errores.clave?.let {
77             Text( text = it, color = MaterialTheme.colorScheme.error)
78         }
79     },
80     modifier = Modifier.fillMaxWidth()
81 )
```

- Al hacer clic en el botón, si todo es válido, navegar a "resumen" usando `navController.navigate("resumen")`.

```
82
83 // Campo dirección
84 OutlinedTextField(
85     value = estado.direccion,
86     onValueChange = viewModel::onDireccionChange,
87     label = { Text(text = "Dirección") },
88     isError = estado.errores.direccion != null,
89     supportingText = {
90         estado.errores.direccion?.let {
91             Text(text = it, color = MaterialTheme.colorScheme.error)
92         }
93     },
94     modifier = Modifier.fillMaxWidth()
95 )
96
97 // Checkbox: aceptar términos
98 Row(verticalAlignment = Alignment.CenterVertically) {
99     Checkbox(
100         checked = estado.aceptaTerminos,
101         onCheckedChange = viewModel::onAceptarTerminosChange
102     )
103     Spacer(Modifier.width(width = 8.dp))
104     Text(text = "Acepto los términos y condiciones")
105 }
106
107 // Botón: enviar
108 Button(
109     onClick = {
110         if (viewModel.validarFormulario()) {
111             navController.navigate(route = "resumen")
112         }
113     },
114     modifier = Modifier.fillMaxWidth()
115 ) {
116     Text(text = "Registrar")
117 }
118 }
119 }
```

◇ Parte 4 – Pantalla de Resumen

- Crear ResumenScreen que observe el mismo UsuarioViewModel y muestre los datos del formulario ingresado en tiempo real usando collectAsState().

```
3 import androidx.compose.foundation.layout.Column
4 import androidx.compose.foundation.layout.padding
5 import androidx.compose.material3.MaterialTheme
6 import androidx.compose.material3.Text
7 import androidx.compose.runtime.Composable
8 import androidx.compose.runtime.collectAsState
9 import androidx.compose.runtime.getValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.unit.dp
12 import com.example.ejemplo3.viewmodel.UsuarioViewModel
13
14 2 Usages
15 @Composable
16 fun ResumenScreen(viewModel: UsuarioViewModel) {
17     val estado by viewModel.estado.collectAsState()
18
19     Column(Modifier.padding(all = 16.dp)) {
20         Text(text = "Resumen del Registro", style = MaterialTheme.typography.headlineMedium)
21         Text(text = "Nombre: ${estado.nombre}")
22         Text(text = "Correo: ${estado.correo}")
23         Text(text = "Dirección: ${estado.direccion}")
24         Text(text = "Contraseña: ${"*".repeat(n = estado.clave.length)}")
25         Text(text = "Términos: ${if (estado.aceptaTerminos) "Aceptados" else "No aceptados"}")
26     }
27 }
```

◇ Parte 5 – Navegación

- Crear archivo AppNavigation.kt con rutas: "registro" y "resumen".

```
3 import androidx.compose.runtime.Composable
4 import androidx.lifecycle.viewmodel.compose.viewModel
5 import androidx.navigation.compose.NavHost
6 import androidx.navigation.compose.composable
7 import androidx.navigation.compose.rememberNavController
8 import com.example.ejemplo3.ui.screen.RegistroScreen
9 import com.example.ejemplo3.ui.screen.ResumenScreen
10 import com.example.ejemplo3.viewmodel.UsuarioViewModel
11
12 2 Usages
13 @Composable
14 fun AppNavigation() {
15     val navController = rememberNavController()
16
17     // 🔥 Aquí creamos el ViewModel una sola vez
18     val usuarioViewModel: UsuarioViewModel = viewModel()
19
20     NavHost(
21         navController = navController,
22         startDestination = "registro"
23     ) {
24         composable(route = "registro") {
25             RegistroScreen(navController, usuarioViewModel)
26         }
27         composable(route = "resumen") {
28             ResumenScreen(usuarioViewModel)
29         }
30     }
31 }
```

- Usar NavHost y NavController.
- Recuerda modificar tu MainActivity para iniciar la navegación



```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.ui.Modifier
import com.example.ejemplo3.navigation.AppNavigation
import com.example.ejemplo3.ui.theme.Ejemplo3Theme

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Ejemplo3Theme {
                Scaffold { innerPadding ->
                    // 📌 Aquí insertamos el flujo de navegación en lugar de Greeting()
                    Box(modifier = Modifier.padding(paddingValues = innerPadding)) {
                        AppNavigation()
                    }
                }
            }
        }
    }
}
```

◇ Parte 6 – Evidencia de trabajo colaborativo

- Una tarjeta en Trello llamada Registro con ViewModel, con checklist del flujo y avance
- Un commit en GitHub con el mensaje: "Formulario y resumen funcional"
- Una captura de pantalla de ResumenScreen con los datos ingresados
- Subir los cambios a una rama feature/registro-formulario